

Hybrid Retrieval RAG Experiments

Overview

Hybrid retrieval combines two fundamentally different retrieval signals – dense semantic search and sparse lexical search – to improve the coverage and relevance of the passages returned for a given query. Dense retrieval, as used in all preceding experiments, encodes both the query and documents into a continuous vector space and retrieves chunks by proximity. It captures semantic relationships and handles paraphrase well but can miss passages that contain the exact keywords of the query. Sparse retrieval, in contrast, operates on term frequency statistics and is well suited to keyword-heavy queries or highly specific terminology, but it does not generalise across synonyms or conceptual reformulations.

For biomedical QA, both failure modes are relevant. A question about a rare gene variant may require precise keyword matching that dense retrieval misses; a question about treatment outcomes may require semantic understanding that sparse retrieval cannot provide. Combining the two signals aims to capture the strengths of each.

Two hybrid configurations were evaluated, both using $k = 5$ and ChromaDB as the dense backend. They differ in how the candidate chunks retrieved by each signal are merged and ranked before being passed to the generator.

Components

Dense Retriever

The dense retriever is the same cosine similarity retriever used in all preceding experiments: queries and stored chunks are encoded using `sentence-transformers/all-MiniLM-L6-v2`, and the top-k chunks by cosine similarity are returned from ChromaDB.

Sparse Retriever (BM25)

The sparse retriever uses BM25 (Best Match 25), a probabilistic retrieval model based on term frequency and inverse document frequency. The full chunk corpus (all 2,849 documents) is loaded into memory as a list of LangChain Document objects and indexed by LangChain's `BM25Retriever`. BM25 scores each chunk by how frequently the query terms appear in it, weighted by how rare those terms are across

the entire corpus. For biomedical text, this is particularly effective at surfacing chunks that contain the precise clinical terminology or abbreviations used in the question.

Experiment 7: Hybrid Retrieval with Reciprocal Rank Fusion

Pipeline

The first hybrid configuration combines the dense and sparse retrieval results using Reciprocal Rank Fusion (RRF). The pipeline proceeds as follows:

1. The question is passed to both the dense retriever (cosine similarity, $k = 5$) and the sparse retriever (BM25, $k = 5$) independently.
2. The two ranked lists of retrieved chunks are fused using RRF. Each chunk is assigned an RRF score based on its rank in each list: a chunk ranked at position r contributes a score of $1 / (k + r)$ to its total, where $k = 60$ is a smoothing constant that reduces the dominance of top-ranked items.
3. Duplicate chunks (identified by their pubid and chunk index metadata) are merged, accumulating RRF scores across both lists.
4. The final ranked list, sorted by descending RRF score, is truncated to the top- k chunks and passed to the generation step.

RRF was chosen over score normalisation approaches (such as weighted linear combination) because it does not require calibrating the score scales of the two retrievers against each other. BM25 scores and cosine similarity scores have different ranges and distributions; RRF sidesteps this problem entirely by operating only on rank positions.

The generation step is identical to the naive RAG configuration: the top- k retrieved chunks are concatenated into a context block and passed to Llama 3.3 70B via the same RAG prompt.

The pipeline additionally records latency and token usage per query. The average end-to-end time per query across the 200-sample evaluation set was **4.24 seconds**, and the average total token usage was **1804.78 tokens per query**.

Parallelisation

The same multi-key ThreadPoolExecutor strategy is used as in all prior experiments. The retriever tuple (dense, sparse) is passed to each thread at construction time. Both retrievers are read-only, so sharing them across threads is safe. Each thread's slice proceeds sequentially with a configurable delay between rows.

Results

Metric	Score
NonLLMContextRecall	0.1967
NonLLMContextPrecisionWithReference	0.2861
BLEU Score	0.1714
ROUGE-L (F-measure)	0.3004
Deepeval Context Recall	0.9183

All 200 questions were answered successfully in a single parallel run with no rate-limit failures.

Experiment 8: Hybrid Retrieval with Cross-Encoder Re-Ranking

Pipeline

The second hybrid configuration replaces the RRF fusion step with a cross-encoder re-ranker. The pipeline proceeds as follows:

1. The question is passed to both the dense retriever (cosine similarity, $k = 5$) and the sparse retriever (BM25, $k = 5$) independently.
2. The two result lists are concatenated and deduplicated using the same (pubid, chunk index) key as in the RRF experiment.
3. The deduplicated candidate pool is re-ranked by a cross-encoder model. For each query-chunk pair, the cross-encoder jointly encodes both texts and outputs a scalar relevance score. This is considerably more expressive than either cosine similarity or BM25 scoring, as the cross-encoder can model interactions between the query and the passage rather than encoding them independently.
4. The top- k chunks by cross-encoder score are selected and passed to the generation step.

The cross-encoder model used is `cross-encoder/ms-marco-MiniLM-L-6-v2`, a compact model trained on the MS MARCO passage ranking benchmark. It is a BERT-based sequence classifier that takes a concatenated (query, passage) pair as input and produces a relevance score. Being trained on large-scale passage retrieval data, it generalises well to biomedical text for the purposes of re-ranking.

The motivation for cross-encoder re-ranking is that both dense and sparse retrievers encode the query and documents independently, which limits how precisely they can model query-document relevance. The cross-encoder re-ranker operates over the union of both retrievers' output and selects the most

relevant chunks from a broader candidate set, combining the coverage advantages of hybrid retrieval with a more powerful relevance signal at the final selection stage.

The average end-to-end time per query for this configuration was **7.56 seconds** measured across the 200-sample evaluation set and the average total token usage was **1804.65**.

Results

Metric	Score
NonLLMContextRecall	0.2071
NonLLMContextPrecisionWithReference	0.305
BLEU Score	0.183
ROUGE-L (F-measure)	0.3007
Deepeval Context Recall	0.9108
Deepeval Context Precision	0.9404

Summary

The two hybrid experiments address the same core problem from different angles. Hybrid RRF with $k = 5$ from both dense and sparse retrievers produces a merged candidate set and ranks them by a rank-fusion score that requires no calibration between retrieval signals. Hybrid with cross-encoder re-ranking goes further: it retains the broader candidate pool from both retrievers but selects the final k chunks using a joint query-document scoring model that is substantially more expressive than either retriever's own scoring function.

For context in relation to the other experiments:

Configuration	Context Recall	Context Precision	BLEU	ROUGE-L
Naive RAG ($k = 3$)	0.1804	0.2979	0.1853	0.3098
Naive RAG ($k = 5$)	0.2008	0.3067	0.1737	0.2966
Naive RAG ($k = 8$)	0.2137	0.3061	0.1797	0.3051
Hybrid RRF ($k = 5$)	0.1967	0.2861	0.1698	0.2970
Hybrid + Cross-Encoder ($k = 5$)	0.2071	0.305	0.183	0.3007
Vanilla LLM	—	—	0.0757	0.2113

The Hybrid RRF result places it broadly in line with the naive RAG configurations at comparable k. The Hybrid RAG with Cross Encoder takes on an average more time but also performs better than the RRF and also Naive RAG at comparable k